L2 Observability & Monitoring / SRE

Focused Question & Answer Bank — 10 Core Topics

Production framework: Impact → Triage → Metrics/Logs/Traces → Isolate → Mitigate → Recover → RCA → Prevent

1. SLI vs SLO vs SLA + Error Budget

What is an SLI?

Spoken answer: An SLI is the actual measurement of service reliability. For an API, examples are successful request rate, availability, latency, or freshness. I make the SLI measurable and tied to a meaningful user journey.

What is an SLO?

Spoken answer: An SLO is the target for an SLI over a defined period. For example, 99.9 percent availability or a latency target for a defined percentile. I use SLOs to drive monitoring, alerting and reliability decisions.

What is an SLA?

Spoken answer: An SLA is the external or contractual commitment to a customer. SLOs are generally internal engineering targets, while an SLA can have business or contractual consequences.

Explain SLI, SLO and SLA with one example.

Spoken answer: For a payment API, successful requests can be the SLI. The engineering SLO could be 99.9 percent successful requests over a month. The SLA is the contractual commitment to the customer. I monitor the SLI against the SLO and manage the external commitment separately.

What is an error budget?

Spoken answer: The error budget is the amount of unreliability permitted by the SLO. With a 99.9 percent SLO, the remaining 0.1 percent is the budget over the measurement period. I use budget consumption to balance reliability work and feature delivery.

When would you page on an SLO?

Spoken answer: I page when the condition requires immediate human action, such as severe errors or fast SLO burn. I would not page simply because a slow-moving trend is consuming a small amount of budget; that can be a ticket or lower-severity notification.

Why are SLOs better than arbitrary infrastructure thresholds?

Spoken answer: SLOs connect monitoring to customer impact. CPU at 80 percent is not automatically an incident if the application is healthy, while a severe increase in customer-facing errors matters even when CPU is low. Infrastructure thresholds are supporting signals.

2. RED + USE + Golden Signals

What are the four Golden Signals?

Spoken answer: Latency, traffic, errors and saturation. They help me understand request performance, demand, failures and capacity pressure.

What is RED?

Spoken answer: RED stands for Rate, Errors and Duration. It is especially useful for request-driven applications and APIs.

What is USE?

Spoken answer: USE stands for Utilization, Saturation and Errors. It is more infrastructure-oriented and helps identify resource bottlenecks.

RED versus USE?

Spoken answer: For an API I start with RED because I care about request behaviour. For infrastructure I use USE. In production I correlate both because application symptoms and infrastructure causes are connected.

Why use P95/P99 instead of only average latency?

Spoken answer: Average latency can hide tail behaviour. P95 and P99 expose slow requests that can materially affect users even when the average looks healthy.

CPU is 95 percent but errors and latency are normal. Do you page?

Spoken answer: Not automatically. I first check whether that utilization is normal for the workload and whether there is evidence of saturation or customer impact. It may be a capacity warning rather than a page.

Latency is high but CPU is only 30 percent. What do you investigate?

Spoken answer: I check database latency, network calls, external APIs, connection pools, locks, disk I/O, thread pools, queues and rate limits. I use traces to see where request time is actually being spent.

3. Prometheus Architecture + PromQL

Explain Prometheus architecture.

Spoken answer: Prometheus is primarily pull-based. Targets expose metrics, Prometheus discovers and scrapes them, and stores time-series data with labels. PromQL queries the data and alert rules evaluate conditions. Alertmanager handles grouping, deduplication and routing.

What are Counter, Gauge, Histogram and Summary?

Spoken answer: Counter is for cumulative events. Gauge is a current value. Histogram represents a distribution such as latency and is useful for aggregation. Summary calculates quantiles on the client side.

Give a PromQL query for request rate.

Spoken answer: A typical query is `sum(rate(http_requests_total[5m]))`. In production I filter or group by service, region, endpoint or status depending on the investigation.

Give a PromQL query for 5xx error rate.

Spoken answer: I calculate failed request rate divided by total request rate: `sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))`. I adapt the metric and labels to the application.

How do you calculate P95 latency from a histogram?

Spoken answer: I use `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))`. I first verify that the histogram buckets cover the service's latency range.

Prometheus stopped scraping one service. What do you check?

Spoken answer: I check target health and scrape errors, then verify the metrics endpoint, service discovery, labels, ServiceMonitor or scrape configuration, endpoints and network connectivity. I also check recent configuration changes.

What is high cardinality?

Spoken answer: It means too many unique label combinations. Request IDs or user IDs as metric labels can create huge numbers of time series. I keep metric labels bounded and use logs or traces for high-cardinality investigation.

Rate versus increase?

Spoken answer: Rate gives a per-second rate of increase over a range and is useful for throughput and alerting. Increase estimates the total increase during the range.

4. Grafana Dashboard & Alerting Design

How would you design a production Grafana dashboard?

Spoken answer: I put availability, traffic, errors and P95/P99 latency at the top. Then I show saturation, dependencies, infrastructure, deployment markers and regional comparisons. I add useful filters such as environment, service and

region. The dashboard should quickly tell the on-call engineer what is broken, who is affected and where to investigate.

What should be on an API dashboard?

Spoken answer: Request rate, success and error rate, P50/P95/P99 latency, saturation, active requests, dependency latency, database health, pod or instance health and deployment information. I also include business-critical SLIs where available.

Grafana says No Data. What do you check?

Spoken answer: I first determine whether the underlying data source has data. If Prometheus has no data, I investigate scraping or instrumentation. If Prometheus has data, I check the Grafana data source, query, time range, variables and dashboard configuration.

Dashboard is green but customers report failures. What does that tell you?

Spoken answer: It may mean the dashboard does not represent the customer journey adequately. I validate metric definitions and aggregation, then check region, endpoint, client and dependency-level signals.

Dashboard has 200 panels. Is that good?

Spoken answer: Not necessarily. I separate service-health dashboards from deep troubleshooting dashboards. The main dashboard should prioritise signals needed for decisions, with drill-down views for detail.

What makes a good alert?

Spoken answer: A clear condition, severity, owner, routing, evaluation window and runbook. Most importantly, it should require action. I prefer alerts based on user impact or SLO risk rather than paging on every infrastructure threshold.

5. Alert-Noise Reduction

You receive 40 alerts per day and 75 percent are non-actionable. What do you do?

Spoken answer: I analyse several weeks of alert history and classify alerts into actionable, informational and noise. I identify poor thresholds, short evaluation windows, duplicate alerts and self-resolving conditions. I tune thresholds and durations, use grouping and deduplication, change severity and routing, and remove alerts that provide no operational value. Then I verify genuine incidents are still detected.

Why is alert fatigue dangerous?

Spoken answer: When engineers receive too many low-value alerts, they can become desensitised and important pages may be delayed. I optimise for actionable signal rather than maximum alert volume.

What is alert grouping?

Spoken answer: Grouping combines related alerts into one incident. If a database outage affects 20 services, I want one coordinated incident with useful context rather than 20 separate pages.

What is deduplication?

Spoken answer: Deduplication prevents repeated copies of the same condition from creating multiple notifications.

Symptom alert versus cause alert?

Spoken answer: A symptom alert represents impact, such as high error rate. A cause signal could be CPU saturation. I generally page on actionable symptoms and use cause signals for diagnosis or lower-severity notification.

An alert fires every five minutes and auto-resolves. What do you do?

Spoken answer: I determine whether human action is required and whether it correlates with impact. If it is transient noise, I can increase the evaluation window, use an SLO or burn-rate condition, lower severity or remove paging.

How do you choose 80 versus 90 percent CPU?

Spoken answer: I don't choose arbitrarily. I use historical baseline, workload behaviour, capacity and correlation with customer-facing SLIs. If latency starts degrading around 80 percent, that supports an actionable threshold; otherwise it may only be informational.

6. OpenTelemetry End-to-End Tracing

What is OpenTelemetry?

Spoken answer: OpenTelemetry is an open-source framework for generating and collecting telemetry such as traces, metrics and logs. I particularly use it for distributed tracing to follow a request across services.

Explain an end-to-end trace.

Spoken answer: A request enters the system with trace context. Each service creates spans and propagates context to downstream calls. Spans are exported through the telemetry pipeline to a tracing backend. I use the trace to identify where time is spent and correlate it with logs and metrics.

How do you troubleshoot a slow API using tracing?

Spoken answer: I find a slow trace, inspect spans and identify the largest contributor to total duration. Then I correlate the trace ID with logs and Prometheus metrics. Once I isolate the bottleneck, I mitigate and validate the user-facing latency.

What is trace context propagation?

Spoken answer: It carries trace and span context across service boundaries so one transaction can be followed end to end.

Metrics show high latency but traces look normal. Why?

Spoken answer: I check whether tracing covers the complete transaction, whether sampling hides affected requests, and whether latency exists outside application spans such as ingress, gateway, client or network time.

What is sampling?

Spoken answer: Sampling controls how many traces are retained or exported. It reduces cost and overhead, but excessive sampling can hide rare incidents. I ensure important errors and slow transactions remain observable.

7. ELK Architecture + Basic KQL

Explain ELK.

Spoken answer: ELK stands for Elasticsearch, Logstash and Kibana. Logstash commonly ingests and transforms logs, Elasticsearch stores and indexes them, and Kibana provides search and visualisation.

How would you investigate an error in Kibana?

Spoken answer: I filter by time, service, environment and severity, then narrow using status, exception or correlation ID. I correlate timestamps and trace or request IDs with metrics and traces.

What is KQL?

Spoken answer: Kibana Query Language is used to filter and search documents in Kibana. For example, `log.level : "ERROR"` or `service.name : "payment" and http.response.status_code >= 500`, depending on the available fields.

KQL versus Lucene?

Spoken answer: KQL is designed for readable filtering in Kibana. Lucene query syntax is another search option. I use the query language supported by the team's Kibana configuration.

Logs are missing in Kibana. What is your flow?

Spoken answer: I trace the pipeline from application or agent to Logstash or collector, Elasticsearch and finally Kibana. I check ingestion errors, parsing, timestamps, network connectivity, index availability, data views and time range.

Elasticsearch disk reaches 90 percent. What do you check?

Spoken answer: I check index growth, retention, shard and replica usage, cluster health and disk watermarks. I identify high-volume sources and avoid deleting data blindly. Mitigation can involve eligible old-index cleanup, retention changes, log-volume control or capacity expansion.

You haven't worked extensively with ELK. How do you answer?

Spoken answer: I would be transparent: my strongest hands-on logging experience is with Splunk and Azure Log Analytics. I understand ELK architecture and the operational workflow, and I can apply the same SRE troubleshooting principles while learning the team's implementation.

8. P1/P2 Incident Walkthrough

Walk me through a P1 incident.

Spoken answer: I acknowledge the incident and establish impact and scope. I bring in the right teams, assign ownership and establish communication. I use metrics, logs and traces to identify the failing layer while keeping service restoration as the immediate priority. Depending on the situation I may rollback, scale, fail over or restore a dependency. After recovery I validate SLIs, document the timeline, perform RCA and track preventive actions.

What is your first priority during P1?

Spoken answer: Understanding customer impact and restoring service safely. Root-cause investigation is important, but I don't delay mitigation while trying to fully explain the problem.

Mitigation versus resolution?

Spoken answer: Mitigation restores or stabilises service without necessarily fixing the underlying cause, such as rollback or scaling. Resolution addresses the root cause. I mitigate first and then drive the permanent fix through the normal change process.

How do you decide P1 versus P2?

Spoken answer: I follow the organisation's documented severity matrix, considering customer impact, scope, criticality, duration and business impact. I don't invent a percentage threshold when a formal definition exists.

What do you communicate during a P1?

Spoken answer: I communicate impact, affected services or regions, current status, mitigation being attempted, next update time and ownership. I avoid speculative root-cause statements until evidence supports them.

What should an RCA contain?

Spoken answer: Impact, timeline, detection, technical root cause, contributing factors, mitigation, recovery and corrective or preventive actions. Actions should have owners and measurable completion criteria.

9. Application Latency Troubleshooting

Checkout API is slow but infrastructure dashboards are green. What do you do?

Spoken answer: I start from the user transaction. I check endpoint-specific RED signals and P95/P99 latency, then inspect OpenTelemetry traces for slow spans. I correlate logs and investigate database, cache, gateway, network and third-party dependencies. I compare recent deployments and regional behaviour, then mitigate and validate the same user-facing SLI.

How do you distinguish CPU saturation from database slowness?

Spoken answer: I correlate latency with both CPU and database telemetry. If CPU saturation aligns with increased application processing time, CPU may be causal. If traces show database spans consuming most request time, I investigate query latency, locks, connection pools and database pressure.

Latency is high only in one region. What do you check?

Spoken answer: I compare the affected region with healthy regions and inspect traffic, errors, latency, DNS, load balancing, networking, dependencies and deployment versions. I filter traces and logs by region.

Latency increases immediately after deployment. What do you do?

Spoken answer: I correlate deployment timing with the SLI degradation, compare versions, inspect traces and logs, and check dependency changes. If evidence points to the release and rollback is safe, I roll back first, then perform RCA and improve testing or deployment gates.

Latency is high but error rate is normal. Is that an incident?

Spoken answer: It can be. A service can successfully return responses while violating a latency SLO. I check latency percentiles and the affected user journey rather than relying on error rate alone.

Users report intermittent failures but error rate looks normal. What do you check?

Spoken answer: I segment by endpoint, region, instance, status code, client and dependency. I compare affected and healthy requests and inspect traces and logs for timeouts, retries or one unhealthy instance.

How can retries make an incident worse?

Spoken answer: Retries can amplify traffic against an overloaded dependency. I check retry rates and backoff behaviour and consider timeouts, circuit breakers, bounded retries and fallback behaviour.

10. Kafka Lag / Consumer / Producer Troubleshooting

Explain Kafka at a high level.

Spoken answer: Producers publish messages to topics, topics are divided into partitions, brokers store the data, and consumers read messages while tracking offsets. Consumer groups allow multiple consumers to process partitions in parallel.

What is consumer lag?

Spoken answer: Consumer lag represents how far a consumer or consumer group is behind the latest available messages. Increasing lag means consumers are not keeping up or cannot progress normally.

Consumer lag suddenly increases. What do you check?

Spoken answer: I determine whether it is one consumer group, topic or partition. Then I check consumer health, processing rate, restarts and errors, partition assignment and broker health. I compare producer throughput with consumer throughput and investigate downstream dependencies.

Producer throughput suddenly doubles. What could happen?

Spoken answer: Consumers may fall behind, causing lag and increased end-to-end latency. I check partition capacity, consumer throughput, broker resources and downstream dependencies. Scaling consumers can help only if there are enough partitions for parallelism.

Consumer is healthy but lag is increasing. Why?

Spoken answer: The process can be healthy while processing too slowly. I check processing time, downstream calls, database latency, CPU, memory, thread pools and message size. I compare incoming rate with processing rate.

One Kafka partition has much higher lag than others. What does that suggest?

Spoken answer: It can indicate partition skew, uneven key distribution, a slow consumer assigned to that partition, or a hot key. I inspect partition assignment and message distribution.

What causes duplicate processing?

Spoken answer: At-least-once processing, consumer retries, failures around offset commits and reprocessing can cause duplicates. I check commit and acknowledgement behaviour and prefer idempotent processing where duplicate delivery is possible.

What is backpressure?

Spoken answer: Backpressure occurs when downstream processing cannot keep up with upstream production. Symptoms include queue growth, Kafka lag and increasing latency. I identify the bottleneck, control input if possible, scale processing capacity and ensure retries do not amplify load.

Kafka brokers are healthy but consumers are failing. What do you check?

Spoken answer: I focus on consumer-side logs, connection and authentication configuration, deserialisation errors, downstream dependencies, resource limits, rebalances and consumer-group membership. Broker health alone does not prove the end-to-end pipeline is healthy.

Rapid L2 Memory Sheet

SLI: what I measure. **SLO:** target. **SLA:** external commitment. **Error budget:** allowed unreliability.

Golden Signals: Latency, Traffic, Errors, Saturation.

RED: Rate, Errors, Duration. **USE:** Utilization, Saturation, Errors.

Prometheus: scrape → store time series → PromQL → alert rules → Alertmanager.

Grafana: visualise, correlate and drill down; do not turn every panel into a page.

Alerts: actionable condition + severity + owner + routing + runbook + evaluation window.

OpenTelemetry: instrumentation + context propagation + spans + telemetry pipeline.

ELK: Logstash ingestion/transform → Elasticsearch index/store → Kibana search/visualisation.

P1: establish impact → engage owners → mitigate → validate → RCA → prevent.

Latency: never assume CPU is the cause; correlate application, dependency, database and network telemetry.

Kafka: producer → topic/partition → broker → consumer group → offset. For lag, compare producer rate with consumer processing rate.